

Check List da Etapa 3

1 - Empacotamento

- a Etapa 3 será utilizada apenas para que o aluno possa verificar a sua progressão para realizar a Etapa 4, que será avaliada como Prova 2, sendo que a sua submissão de envio poderá ser verificada desde que siga rigorosamente todos os procedimentos de Empacotamento e a metodologia ilustrada nos Tutoriais disponibilizados

1.1 - Nome do diretório zipado

- **procedimento**
 - copiar o string abaixo e somente alterar o nome do aluno para o seu nome completo, respeitando os espaçamentos e o padrão de letras maiúsculas e minúsculas
 - LPI - Etapa 3 - Alberto Luis da Silva Torres

1.2 - Conteúdo do arquivo zipado

- **procedimento**
 - incluir somente : o diretório src e os arquivos espec.pdf e fontes.pdf

1.3 - Diretório para Comprovação da Execução do Sistema : src

- **procedimento**
 - copiar somente o diretório src do projeto
 - com seus diretórios padronizados (conforme o sistema de referência do Tutorial da Etapa) e seus arquivos Python

1.4 - Arquivos pdf para Comprovação da Etapa da Prova Escrita : espec - fontes

1.4.1 - Cabeçalho de cada arquivo pdf

- **procedimento**
 - copiar o string abaixo e só alterar o nome do aluno para o seu nome completo, respeitando os espaçamentos e o padrão de letras maiúsculas e minúsculas
 - Linguagem de Programação I - Etapa 3 - Alberto Luis da Silva Torres
 - verificar se os 2 arquivos pdf (espec e fontes) contém o mesmo cabeçalho

1.4.2 - Final do arquivo de cada arquivo pdf : cidade, data -- assinatura

- **procedimento**
 - utilizar a data de envio da Etapa, que deverá pertencer ao período previsto para a Etapa no Plano de Ensino
 - verificar se os 2 arquivos pdf (espec e fontes) contém no final do arquivo:
 - cidade, dd/mm/aaaa
 - assinatura
 - caso a assinatura não seja digital (portal gov.br)
 - utilizar o nome por extenso com letra legível
 - assinar em um papel com letra cursiva bem legível e inserir a imagem no Adobe Reader para utilizar como assinatura -- não utilizar o mouse para assinatura porque a letra fica ilegível

1.4.3 - Conteúdo do arquivo : espec.pdf

- **procedimento**
 - será considerada inválida a Etapa
 - cujo arquivo espec.pdf não utilizar somente as entidades e relacionamentos acordados com o professor
 - para a qual, após o cabeçalho do arquivo espec.pdf, não constarem as seguintes informações, seguindo rigorosamente o exemplo seguinte
 - para as entidades cujo plural não for formado apenas acrescentando o caracter 's' (com é o caso de Maestros e Patrocínios), deverá ser informado, além da entidade no singular, também a entidade no plural (como é o caso de Patrocinadores e PeçasMusicais)
 - a terminação dos plurais em vermelho é somente para chamar atenção no Check List e não deverá ser utilizada na Etapa
 - para as demais informações, que deverão seguir o modelo apresentado na seção 2, será atribuída nota, penalizando as partes do espec que não seguirem as regras definidas na seção 2
 - conforme a proposta de especificação acordada com o profess
 - deve haver somente um relacionamento múltiplo
 - com a cardinalidade [1:n] ou [n:n]
 - a entidade associativa deve referenciar somente duas entidades
 - sendo um delas a principal do relacionamento múltiplo (ex: Repertório)
 - e a outra deverá ser a entidade sem referências, que não participou como entidade secundária do relacionamento múltiplo (ex: Maestro)

Título do projeto: Apresentações de Repertórios Musicais

Entidades

- ApresentaçãoMusical : ApresentaçõesMusicais
- Maestro
- PeçaMusical : PeçasMusicais
- PeçaMusicalClássica : PeçasMusicaisClássicas
- PeçaMusicalPopular : PeçasMusicaisPopulares
- Repertório

Relacionamentos

- Sem Referências : PeçaMusical, Maestro
- Relacionamento Múltiplo : Repertório [n:n] PeçaMusical
- Associação : ApresentaçãoMusical [Repertório, Maestro]
- Herança : PeçaMusical [PeçaMusicalClássica, PeçaMusicalPopular]

1.4.4 - Conteúdo do arquivo : fontes.pdf

- **procedimento**

- incluir o código fonte de cada um dos arquivos do diretório src em uma nova página, na ordem em que eles aparecem no diretório src
 - não pode haver divergência entre o conteúdo do arquivo do diretório src e o arquivo incluído no fontes.pdf
 - utilizar somente o texto (não usar print) com fonte legível
 - ex: Times New Roman ou Arial com fonte 12
 - utilizar fundo branco
- antes de cada arquivo, incluir o cabeçalho em negrito com o caminho do arquivo, precedido do string \$\$\$, conforme os arquivos da Etapa 2 para a especificação ilustrada na seção 1.4.3
 - \$\$\$ src.controle.projeto
 - \$\$\$ src.entidades.apresentação_musical
 - \$\$\$ src.entidades.maestro
 - \$\$\$ src.entidades.peça_musical
 - \$\$\$ src.entidades.repertório
 - \$\$\$ src.util.data
 - \$\$\$ src.util.gerais
- pular uma linha entre o cabeçalho (em negrito) de cada arquivo e o seu conteúdo (sem negrito)
- caso o aluno queira verificar se não esqueceu nenhum conteúdo, antes de realizar a Etapa, poderá pesquisar no Adobe Reader com o string : \$\$\$
 - Localizar texto ou ferramentas - ... - Pesquisa avançada - \$\$\$
 - verificar se todos os arquivos do diretório src foram incluídos no fontes.pdf
 - com cabeçalhos corretos
 - e na ordem que aparecem no diretório src
 - clicar em cada resposta da pesquisa
 - verificar se o cabeçalho de cada arquivo corresponde ao conteúdo do arquivo

2 - Definição da Especificação

- seguir as regras de nomes para :
 - entidade (ex: Maestro, PeçaMusical), atributo (ex: anos_experiência, compositor) e referência (ex: maestro, peça_musical)
- com exceção da entidade principal de associação (para a qual não é definida chave) e das subclasses (que herdam a chave da subclasse), o primeiro atributo das entidades deve ser a sua chave (que identifica unicamente o objeto da entidade) sublinhada (ex: nome)
- a única referência da entidade principal do relacionamento múltiplo deve estar no plural e em negrito (ex: **peças_musicais**)
- as duas referências da entidade associadora devem estar no singular e em negrito (ex: **repertório, maestro**)
- o número mínimo de atributos das entidades devem ser (lembrando que referências não podem ser contadas como atributos)
 - entidade associadora (ApresentaçãoMusical) : deve ter pelo menos um atributo
 - na herança, a superclasse (PeçaMusical) e as subclasses (PeçaMusicalClássica, PeçaMusicalPopular) : devem ter pelo menos dois atributos
 - demais entidades (Maestro, Repertório) : devem ter pelo menos 3 atributos
- a superclasse deve definir os atributos comuns que serão herdados pelas subclasses
 - inclusive a chave que também será herdada
- a subclasse devem definir atributos específicos
 - distintos dos atributos herdados da superclasse
 - e também distintos dos atributos da outra subclasse
- os atributos das entidades devem ser relevantes para o projeto em questão
 - projeto Venda de Veículos : o atributo valor (do veículo) é relevante
 - projeto Manutenção de Veículos : o atributo valor (do veículo) não é relevante
- um atributo não deve carregar o nome da entidade
 - na entidade Filme não deve ser definido o atributo título e não : título_filme
 - com exceção de subclasses quando for necessário diferenciar atributos com mesmo nome
 - como tipo_carro para a subclasse Carro e tipo_caminhão para a subclasse Caminhão
- o atributo de uma entidade pode ser relacionado como outra entidade que não tenha sido definida no projeto
 - se um projeto tiver a entidade AnimalDoméstico e não tiver a entidade Proprietário, então a entidade AnimalDoméstico poderá utilizar o atributo : nome_proprietário
 - caso contrário, a entidade AnimalDoméstico não deverá utilizar esse atributo, que será definido na entidade Proprietário como : nome
- atributos com um pequeno conjunto de valores possíveis deverão ter seus itens definidos em Enumerados
 - um atributo booleano exibe sempre os valores True e False e, portanto, não deve ser definido em Enumerados

O restante do modelo da especificação ilustrado na seção 1.4.3 do Empacotamento, é ilustrado a seguir. Os atributos e enumerados farão parte da avaliação da Etapa. As chaves deverão estar sublinhadas e as referências em negrito. Somente as subclasses não tem chave sublinhada, porque herdam a chave da superclasse. Como única exceção, não deverá ser especificada chave para a entidade associadora (ApresentaçãoMusical).

Atributos e Referências

- ApresentaçãoMusical: data, **repertório**, **maestro**
- Maestro: nome, anos_experiência, estrangeiro
- PeçaMusical: título, compositor, duração, tom
- PeçaMusicalClássica : estilo_música_clássica, muito_conhecida
- PeçaMusicalPopular : estilo_música_popular, instrumentação_característica
- Repertório: título, nome, data_montagem, mesmo_compositor, **peças_musicais**

Enumerados

- estilo_musica_clássica : barroco, romântico, clássico, modernismo
- estilo_musica_popular : pop, rock, country, reggae, blues, jazz
- instrumentação_característica : guitarra elétrica, bateria, baixo, teclado, saxofone, violão, trompete, piano, bateria eletrônica, violino, contrabaixo, sintetizador

3 - Padronização de Nomes na Implementação

3.1 - Nomes de Diretórios e Arquivos

- letras minúsculas com palavras ligadas por underscore
- diretórios utilizam somente nomes padronizados
 - na Etapa 1 : controle, entidades, util
- exemplos
 - no diretório controle, o arquivo : apresentações_repertórios_musicais
 - o nome do arquivo deve englobar as principais palavras do título do projeto dispensando as preposições (não usar: apresentações_**de**_repertórios_musicais)
 - no diretório entidades, os arquivos : maestro, peça_musical
 - no diretório util, os arquivos : data, gerais

3.2 - Nomes das Classes

- palavras com a primeira letra maiúscula e as demais letras minúsculas
 - exemplos : Maestro, PeçasMusicais

3.3 - Nomes de Variáveis, Métodos, Funções e Variáveis

- letras minúsculas, com palavras ligadas por underscore
 - variáveis : data, objetos
 - métodos (funções internas às classes) : calcular_idade
 - métodos padronizados ladeados por 2 underscores : __init__ - __str__
 - funções : cadastrar_peças_musicais

3.4 - Nomes de Conjuntos e Funções relacionadas

- o nome dos conjuntos de entidades deve ser o nome da entidade em letra minúscula no plural
 - entidade : PeçaMusical
 - conjunto : peças_musicais
- os métodos get deve utilizar também os nomes no plural
 - método : get_peças_musicais
- cada método utilizado para inserir um objeto em um conjunto deve utilizar o nome no singular, pois recebe apenas um objeto para inserir
 - método : inserir_peça_musical

4 - Armazenamento de Conjuntos de Objetos

4.1 - Conjuntos de Objetos Globais

- com exceção da entidade secundária de um relacionamento **1:n** (ex: Veículo), as demais entidades (ex: AgênciaPublicidade e Montadora) devem ter seus conjuntos de objetos armazenados em dicionários globais (ex: agências_publicidade e montadoras)
 - inicialmente vazios : { }
 - e preenchidos por funções de inserção --- ex: inserir_agência_publicidade
 - que devem verificar se a chave do objeto já não se encontra no dicionário
- a entidade associadora (ex: ApresentaçãoMusical) deve ser armazenada em lista pois, como referencia pelo menos duas outras entidades, não tem uma chave única
 - inicialmente vazia : []
 - e preenchida por função de criação --- ex: criar_apresentação_musical
 - que recebe as chaves das duas entidades referenciadas (dado que os objetos a serem referenciados já foram criados anteriormente), além dos atributos da entidade associadora, e obtém os objetos a partir de suas chaves
 - então, cria o objeto e executa o método de inserção, que deve verificar se o objeto já não se encontra na lista --- ex: inserir_apresentação_musical

4.2 - Conjunto de Objetos Locais da Entidade Secundária de um Relacionamento **n:n**

- a entidade principal de um relacionamento **n:n** (ex: Repertório) deve armazenar um dicionário local (ex: self.peças_musicais) da entidade secundária (ex: PeçaMusical) que se relaciona com mais de um objeto entidade principal
 - mas o dicionário global com todos os objetos da entidade secundária continua existindo
- para inserir objetos neste dicionário local, utilizar um método de inserção (ex: inserir_peças_musicais (self, chaves_peças_musicais) da classe Repertório) que recebe uma lista de chaves dos objetos da entidade secundária e insere, no dicionário local (self.peças_musicais), o objeto recuperado do dicionário global (atores da classe Ator) a partir de sua chave

4.3 - Conjunto de Objetos Locais da Entidade Secundária de um Relacionamento 1:n

- a entidade principal de um relacionamento **1:n** (ex: Montadora) deve armazenar um dicionário local (ex: self.veículos) da entidade secundária (ex: Veículo) que se relaciona com um único objeto da entidade principal
 - e neste caso, como exceção, o conjunto de objetos globais da entidade secundária (ex: veículos da classe Veículo) deixa de existir
- para inserir objetos neste conjunto, utilizar várias vezes o método de inserção (ex: inserir_veículo (self, veículo) da classe Montadora) que recebe um único objeto e insere no dicionário local

4.4 - Armazenamento dos Objetos das Subclasses

- os dicionários que armazenam os objetos das superclasses, deverão passar a armazenar os objetos das subclasses com seus atributos herdados e os seus atributos específicos

5 - Funcionalidades Solicitadas na Etapa

As Etapas deverão seguir a metodologia do Tutorial 3:

- utilizar a função de impressão e a classe **Data** do diretório **util** exatamente da forma como foram definidas;
- definir as classes das entidades solicitadas na Etapa e seus conjuntos de objetos (com seus métodos de leitura e inserção), e o método de filtragem de objetos da entidade associadora, nos arquivos do diretório **entidades** conforme ilustrados no tutorial;
- definir as subclasses no mesmo arquivo de suas superclasses;
- utilizar o único arquivo do diretório **controle** somente para o cadastro e a impressão das entidades solicitadas na Etapa.

5.1 - Cadastro e Impressão dos Objetos Cadastrados

5.1.1 - Classes a serem cadastradas e impressas

- entidades sem referências (ex: Maestro - PeçaMusical)
- entidade primária do relacionamento múltiplo (ex: Repertório) referenciando objetos da classe secundária do relacionamento (ex: PeçaMusical)
 - pelo menos alguns objetos da classe primária devem referenciar pelo menos dois objetos da classe secundária
- entidade associadora (ex: ApresentaçãoMusical)
- o método `__init__` de cada classe deve checar a validade do valor recebido para atributos enumerados, verificando se esse valor pertence a uma tupla de strings de valores possíveis
- os cadastros dos objetos da superclasse deverão passar a cadastrar os objetos das subclasses, abrangendo os atributos herdados e os atributos específicos

5.1.2 - A primeira linha da impressão de cada classe deve conter

- nome da classe : nomes das colunas que estão sendo mostradas, considerando os atributos das duas subclasses, como no exemplo seguinte
 - PeçaMusical : título, compositor, duração, tom,
clássica [estilo_música_clássica, muito_conhecida]
| popular [estilo_música_popular, instrumentação_característica]
- não formatar os nomes das colunas
 - pois quando o nome da coluna (ex: anos_experiência) é maior que o conteúdo (ex: 12) são gerados vários espaços desnecessários entre as colunas

5.1.3 - As colunas devem estar alinhadas

5.1.4 - Para evitar colunas com espaços vazios desnecessários

- o espaçamento de cada coluna deve ser baseado no tamanho do maior texto impresso na coluna, acrescido de dois espaços (de separação entre colunas)

5.1.5 - Um atributo booleano deve ser informado somente para valor True

- informar na última coluna e utilizar um string mais amigável em vez de True
- exemplo para o atributo : muito_conhecida = True
 - informar : muito conhecida
 - omitir informação para : muito_conhecida = False
 - não informar : não muito conhecida ou pouco conhecida

5.1.6 - Para a classe principal do relacionamento múltiplo

- inicialmente imprimir os objetos da classe principal do relacionamento múltiplo (ex: Repertório)
- imprimir o nome da classe principal do relacionamento e os nomes de seus atributos
 - indentando: o nome da classe secundária do relacionamento e os nomes de seus atributos
 - exemplo
Repertório : título, nome, data_montagem, mesmo_compositor
- PeçaMusical : título, compositor, duração, tom
- loop da classe principal : para cada objeto da classe secundária
 - imprimir o objeto da classe principal e os objetos da classe secundária
 - exemplo
1 - | Mestres da Música Clássica | 10/10/2025 |
- | Fuga em Ré Menor | Johann Sebastian Bach | 10 min | Ré maior |
| barroco | muito conhecida |
- | Rodo a La Turca | Wolfgang Amadeus Mozart | 8 min | Lá maior |
| clássico | muito conhecida |
- | Sonata ao Luar | Ludwig van Beethoven | 7 min | Dó sustenido menor |
| romântico | muito conhecida |

5.2 - Filtragem e Impressão dos Objetos Filtrados

5.2.1 - Os filtros serão definidos para a classe associadora (ex: ApresentaçãoMusical)

- deverá ser definido um filtro para cada classe do projeto (com exceção das subclasses)
- será necessário que cada filtro englobe o nome de sua classe de origem
 - ex: ApresentaçãoMusical --- vou utilizar o negrito somente para chamar a atenção
 - data_mínima_apresentação_musical=Data(1,10,2025),
compositor_peça_musical='Johann Sebastian Bach',
prefixo_nome_maestro='Herbert ', mesmo_compositor_repertório=False,
estilo_música_clássica='barroco',
instrumentação_característica_música_popular='piano'
- pelo fato do objeto da classe associadora (ex: ApresentaçãoMusical) não referenciar um objeto de classe secundária do relacionamento múltiplo (ex: PeçaMusical), todos os objetos da classe secundária referenciados pela classe principal (ex: todos os objetos da classe PeçaMusical referenciados pelo objeto da classe ApresentaçãoMusical), deverão ser testados em relação ao filtro da classe secundária (ex: compositor_peça_musical)

5.2.2 - Após informar os nomes e valores dos filtros utilizados

- informar em cada impressão o nome da classe e os nomes das colunas
 - ex: Maestro: nome, anos_experiência, estrangeiro
- a impressão do nome do filtro deve ser equivalente (string com alguma preposição, se necessária para tornar o nome mais amigável) ao nome do filtro passado como parâmetro para as funções de seleção, que retornam os objetos filtrados
 - ex: parâmetro do filtro : mínimo_anos_experiência
 - imprimir filtro como : mínimo de anos de experiência

5.2.3 - Logo após a impressão dos objetos cadastrados de uma dada classe, filtrar e imprimir

- sem nenhum filtro solicitado : deverá retornar a própria lista de objetos cadastrados
- filtrar acrescentando um filtro por vez, informando o nome e o valor do filtro, **à direita** dos filtros já informados
 - a cada novo filtro, manter os filtros já utilizados, e checar se a lista diminuiu pelo menos de um objeto
 - ao final da filtragem deve restar pelo menos um objeto no resultado

5.2.4 - Filtros numéricos (int - float) deverão utilizados como valores mínimos ou máximos

- ex: int : mínimo_anos_experiência -- float : preço_máximo_ingresso
- cpf, rg e cnpj : devem ser utilizados como strings e não como valores numéricos

5.2.5 - Filtro gerado a partir de valor booleano deverá ser informado para valor True ou False

- ex: atributo : estrangeiro
 - informar para valor True : estrangeiro
 - informar para valor False : não estrangeiro ou nacional

5.2.6 - Não usar como filtros, atributos que identificam um único objeto no conjunto

- atributos que identificam o objeto com único em conjunto de objetos da mesma classe, não podem ser utilizados como filtros
 - identificadores : cpf, título, nome
 - atributos em geral atribuídos a um único objeto : telefone, email
- nestes casos pode ser utilizado o prefixo de atributo
 - ex: prefixo_nome = 'Mar', prefixo_telefone = '67'

6 - Verificação da Etapa 3

- a verificação da Etapa 3 deverá considerar o atendimento às regras de empacotamento e a utilização da metodologia dos tutoriais
- serão verificadas como corretas :
 - somente as partes da Etapa que executarem corretamente, em relação a todas as funcionalidades a serem implementadas
 - e que estiverem de acordo com a especificação (espec.pdf)
 - e que atenderem as regras das seções anteriores para :
 - definição da especificação
 - padronização de nomes na implementação
 - armazenamento de conjuntos de objetos
 - requisitos das funcionalidades solicitadas na Etapa